



UNIVERSIDAD
POLITÉCNICA
DE MADRID



medal



Facultad
de
Informática



UNIVERSIDAD
COMPLUTENSE
MADRID

Introducción al Procesamiento de Lenguaje Natural - Transformers

Álvaro García Barragán

Universidad Complutense de Madrid

Víctor Robles Forcada

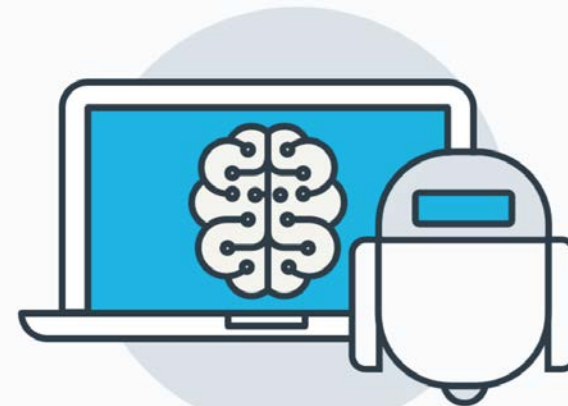
Universidad Politécnica de Madrid

22 de febrero de 2023

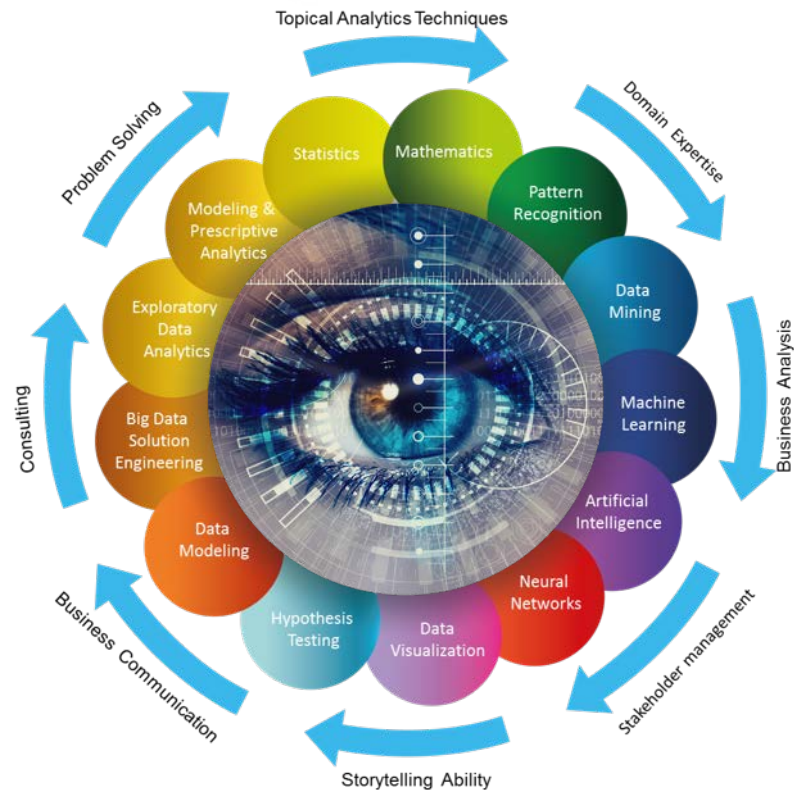


Índice

1. Introducción
2. Embeddings
3. RNN y LSTM
4. Transformers
 - a. Mecanismos de Atención
 - b. Codificación Posicional
 - c. Pre-training & Fine-tuning
5. Research lines



Introducción



“La teoría y el desarrollo de sistemas computacionales para llevar a cabo tareas que normalmente requieren de inteligencia humana, como percepción visual, reconocimiento de voz, toma de decisiones o traducción de lenguajes”
Oxford Living Dictionary

Tareas que requieran de la inteligencia humana

- Predicción de resultados y reconocimiento de patrones basados en datos históricos
- Reconocimiento de eventos anómalos y toma de decisiones
- La interpretación de información visual
- Comprensión del lenguaje y participación en conversaciones
- Extracción de información de orígenes para obtener conocimientos



Cargas de trabajo comunes en inteligencia artificial



UNIVERSIDAD
POLITÉCNICA
DE MADRID

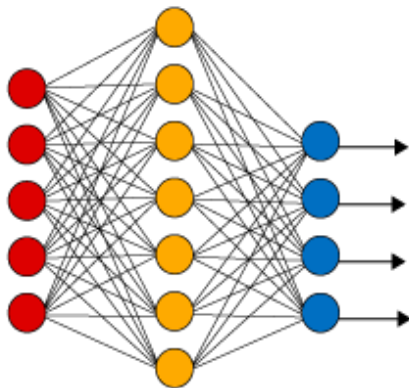


	Aprendizaje automático	Modelos predictivos basados en datos: la base de la IA
	Detección de anomalías	Sistemas que detectan patrones o eventos inusuales, lo que permite una acción preventiva
	Visión por computador	Aplicaciones que interpretan la entrada visual de cámaras, imágenes o vídeos
	Procesamiento de lenguaje natural	Aplicaciones que pueden interpretar el lenguaje escrito o hablado, y participación en diálogos con usuarios humanos

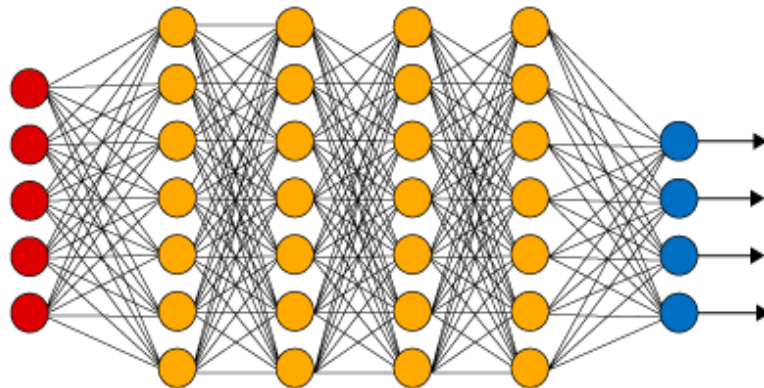
Deep Learning

Una forma de aprendizaje automático basada en los mismos principios que las redes neuronales del cerebro

Simple Neural Network



Deep Learning Neural Network



● Input Layer

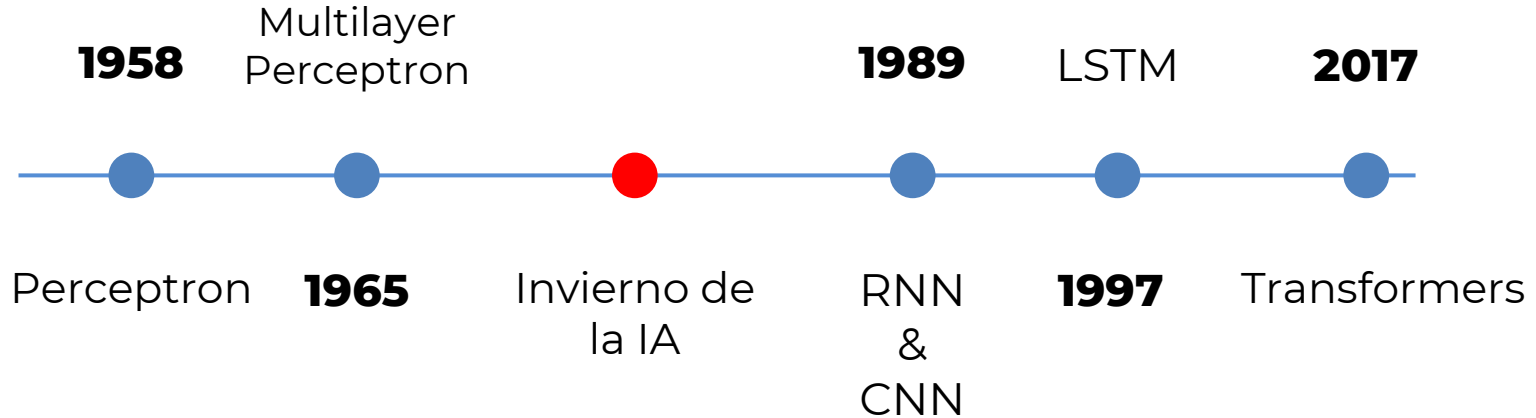
● Hidden Layer

● Output Layer

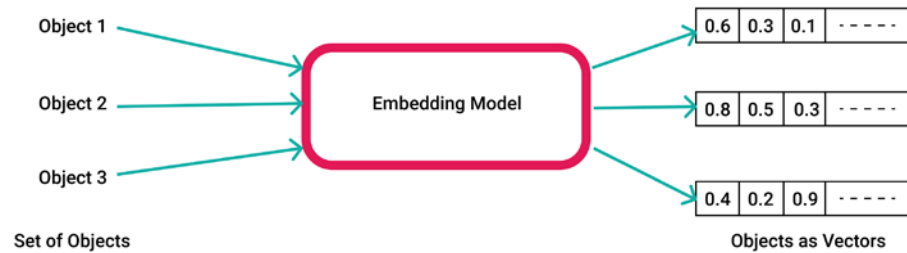
Historia del NLP



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Embeddings



Tokens

¿Cómo se pueden meter palabras en un Red Neuronal?
Tokenización + Embeddings

Token - Unidad mínima de codificación

División por palabras

División por sub palabras

División por caracteres

BTE (Byte pair encoding) (GPT)

In : Aaabdaaabac

Y=ab

Z=aa

X=ZY

Out : XdXac

Necesario conversión de los token a números
(codificación y decodificación)

Tipos de codificación

- integer-encoding
- one-hot-encoding
- word-embeddings

Tipos de Codificación



UNIVERSIDAD
POLITÉCNICA
DE MADRID



INTEGER ENCODING

Perro = 1

Gato = 2

Mapache = 30

ONE-HOTE-ENCODING

Perro = [1 0 0]

Gato = [0 1 0]

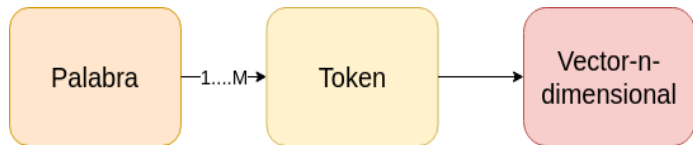
Mapache = [0 0 1]

Integer-encoding: ¿La palabra mapache dista de perro en 29 unidades?

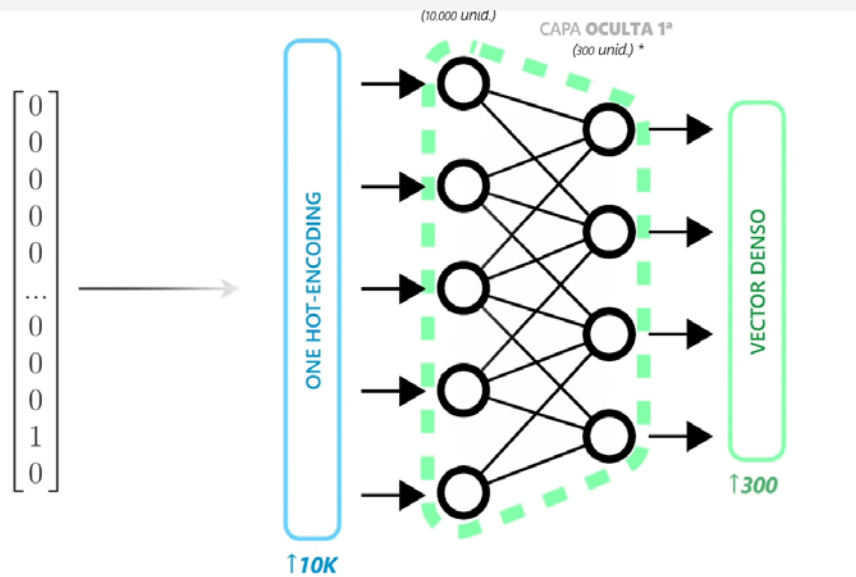
One-hot-encoding: vocabulario de 10000 palabras necesitamos 10000 vectores de 10000 componentes
¿Cada palabra dista lo mismo del resto?

Embeddings

Convertir representación one-hot-encoding a representación más compacta



Solución: word embeddings que expresen la relación semántica del lenguaje



Embeddings: Modelos de distribución de palabras

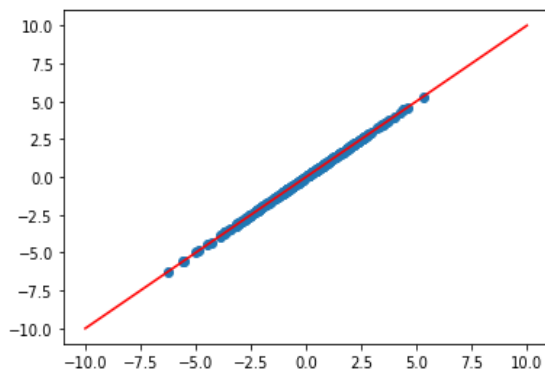
- Google 2013 -> **word2vec** (modelo pre-entrenado que codifica los tokens a vectores)
- Operaciones en espacio vectorial
- Palabras similares tienen vectores próximos en el espacio

Reducción de dimensionalidad (PCA) 300->3, <http://projector.tensorflow.org/>

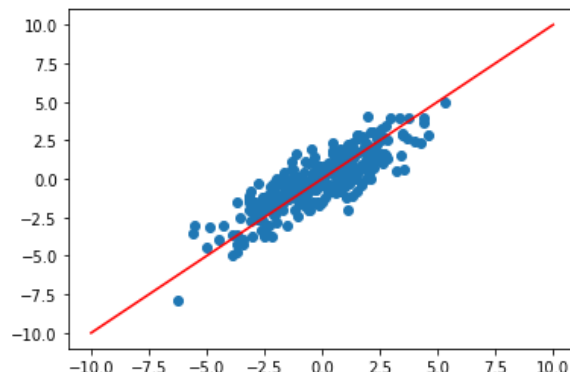
Similitudes

- Que similitudes hay entre perro y gato (animales, mascotas y sustantivos)
- Similitud entre palabras en vectores de 300 elementos , grafico de dispersión

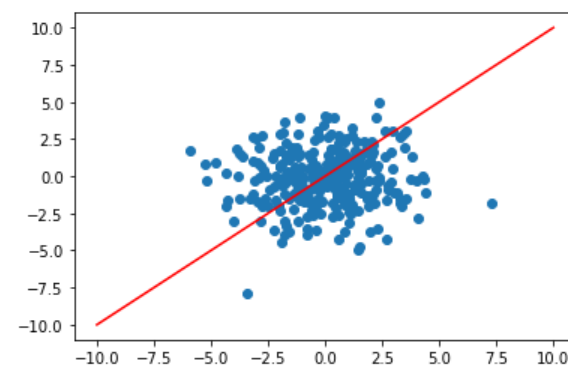
Word1: gato
Word2: gato
Similitud: 1.0



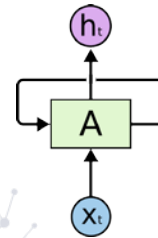
Word1: gato
Word2: perro
Similitud: 0.8487285375595093



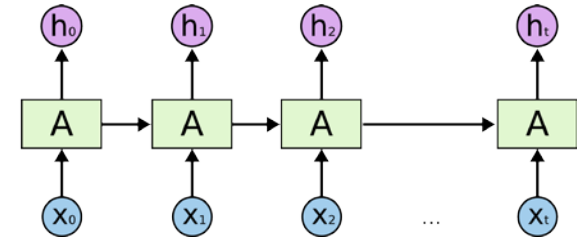
Word1: hola
Word2: perro
Similitud: 0.05708715319633484



RNN y LSTM

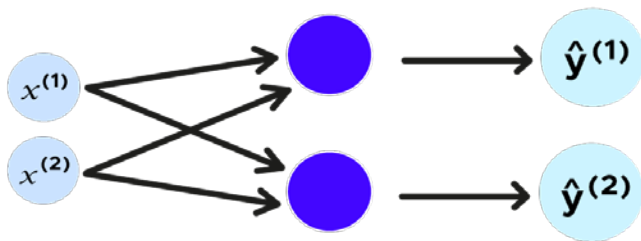


=



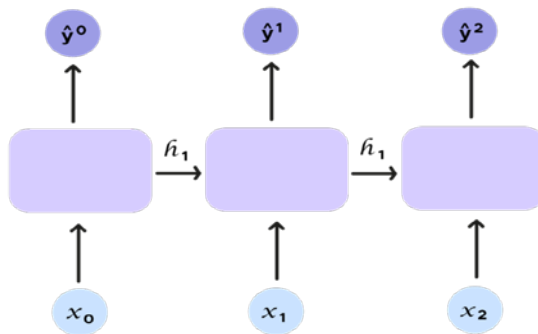
Multilayer vs RNN

Multilayer



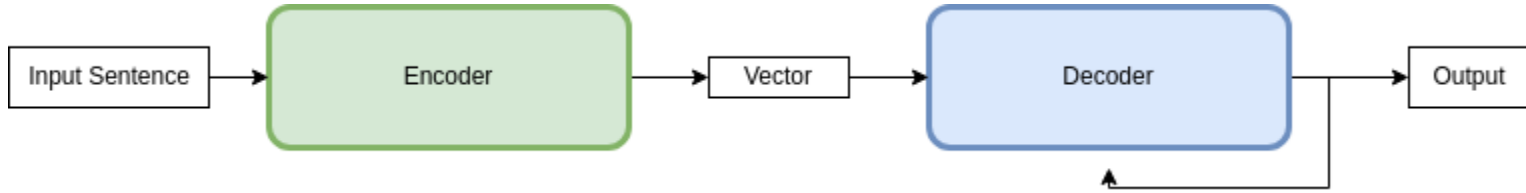
Cada token es independiente

RNN

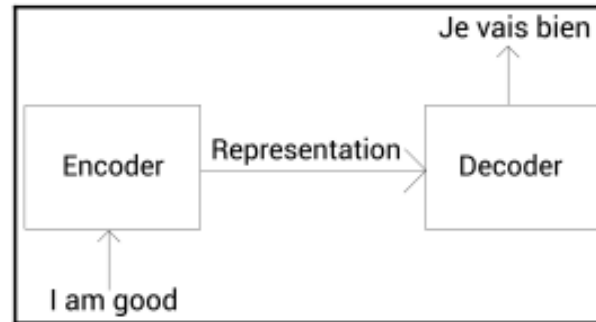


Cada token depende de los anteriores

Modelos codificador-decodificador - Seq2Seq

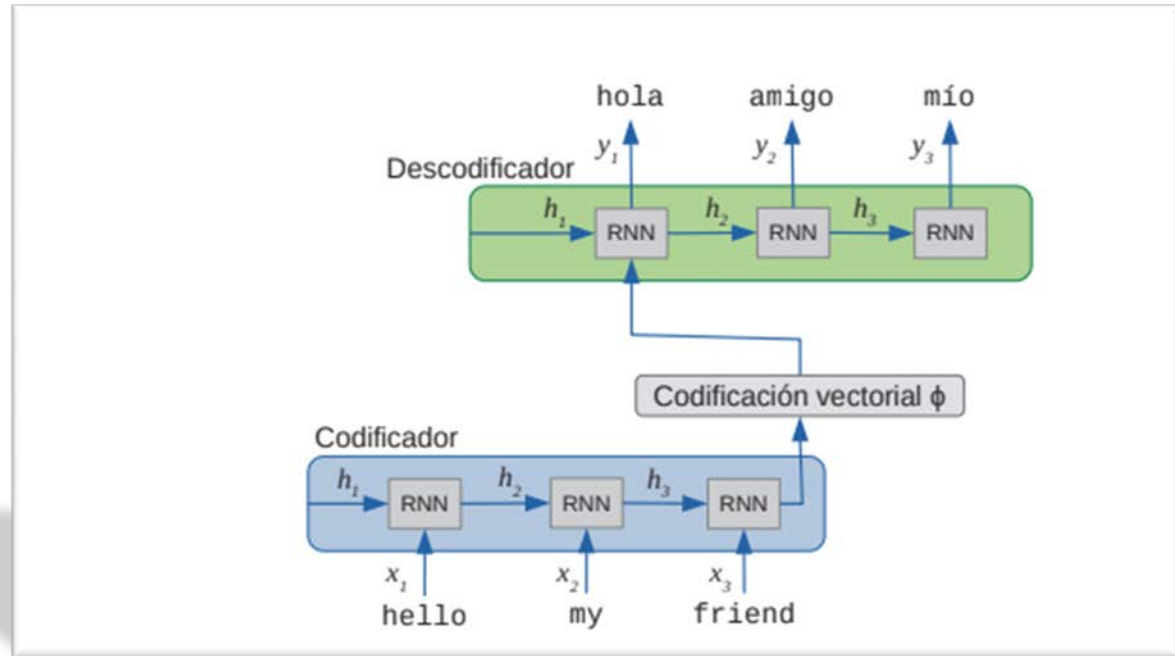


Todos los modelos que se presentan a continuación siguen este esquema



Ejemplo de RNN traducción de inglés a español

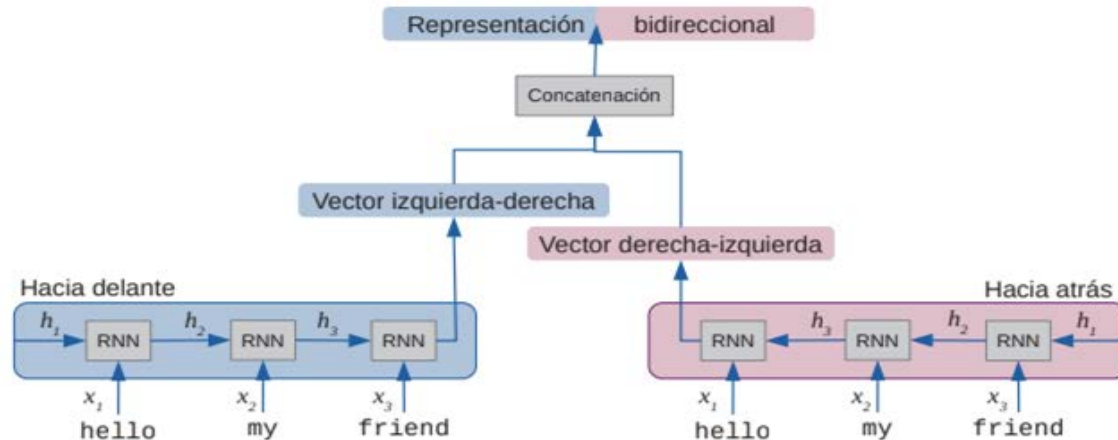
Ejemplo
modelo Seq2Seq de
traducción con RNN



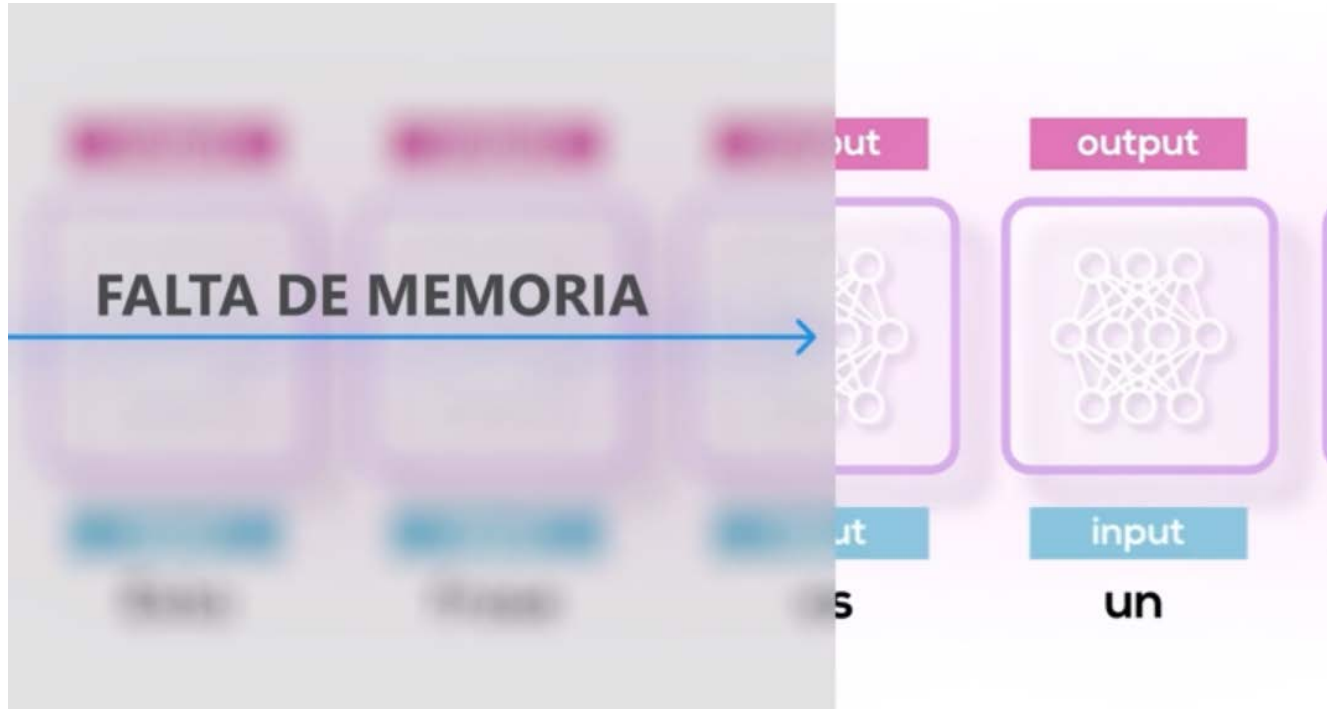
Modelos bidireccionales recursivos

Necesario
bidireccionalidad
para entender el
texto

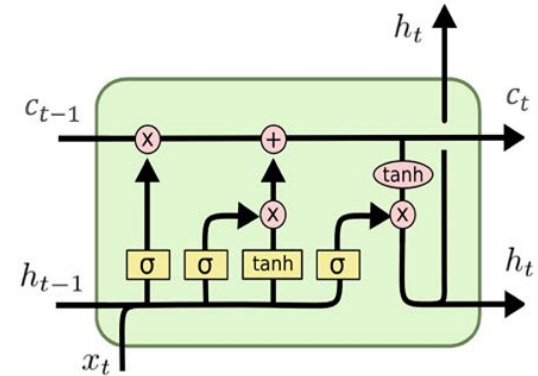
Si lo necesitas, te puedo enviar el currículum
Te puedo enviar el currículum si lo necesitas



Problema de la falta de memoria - desvanecimiento del gradiente en RNN



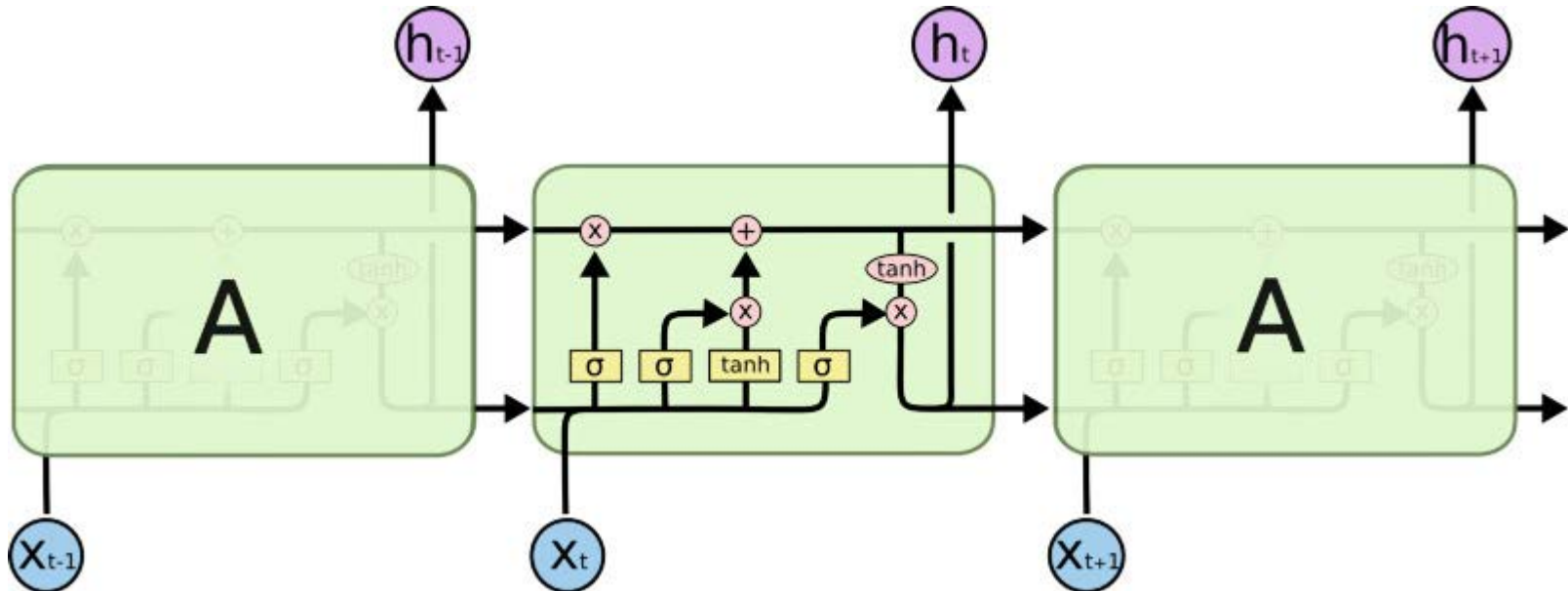
LSTM - Long Short-Term Memory Networks



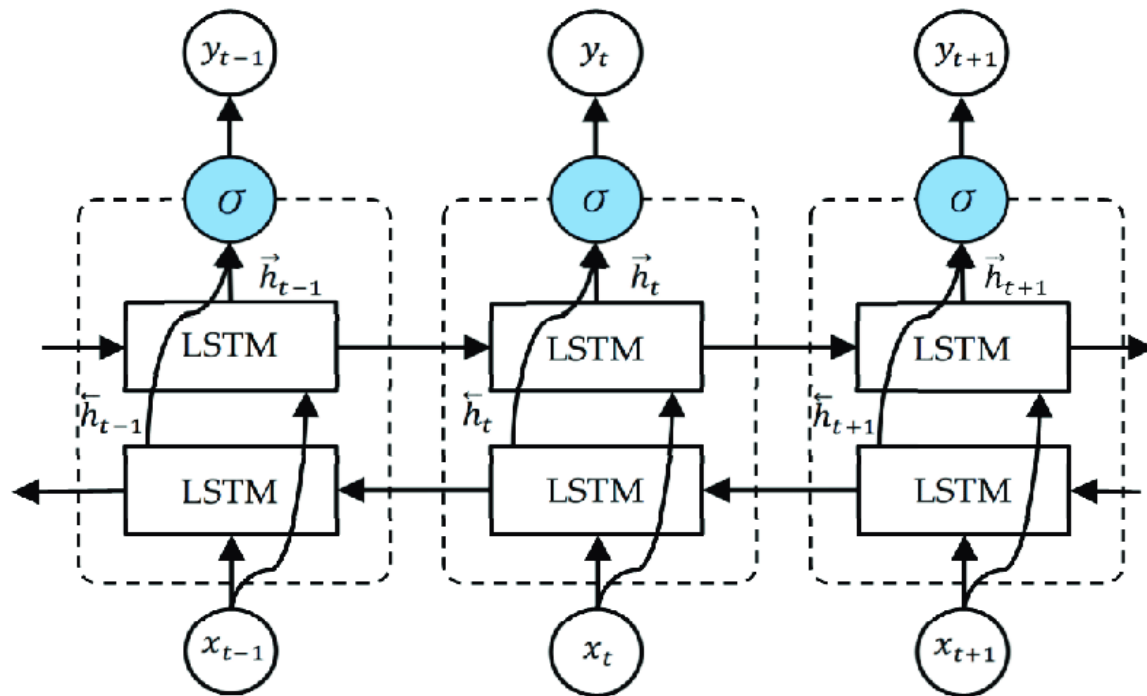
Pasamos de dos entradas a tres

- Token actual (x_t)
- Salida celda anterior (h_{t-1})
- Memoria (c_{t-1})

LSTM - Long Short-Term Memory Networks

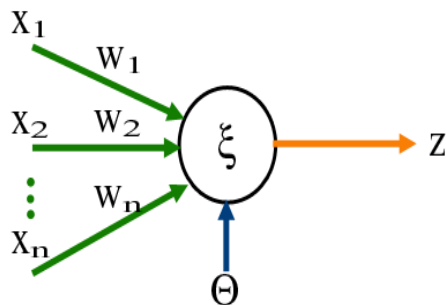


BILSTM

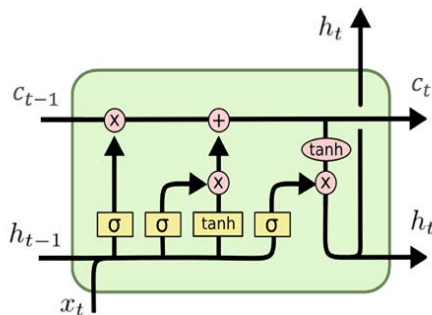


Comparación

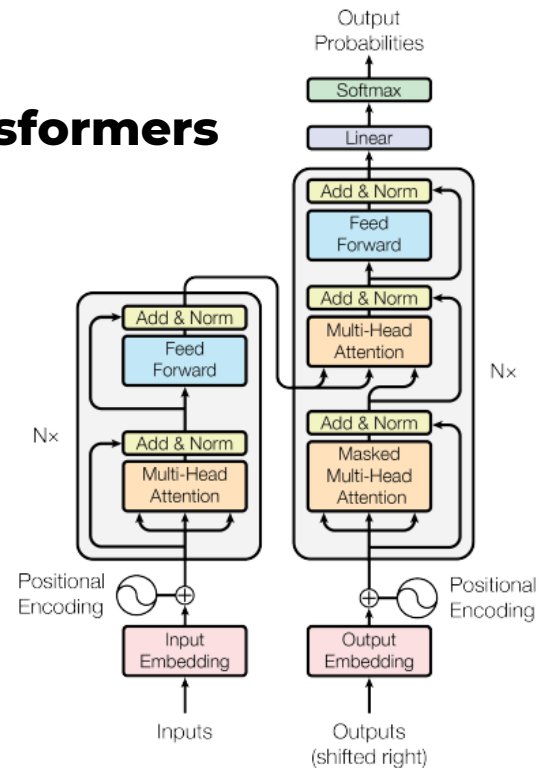
Perceptron



LSTM



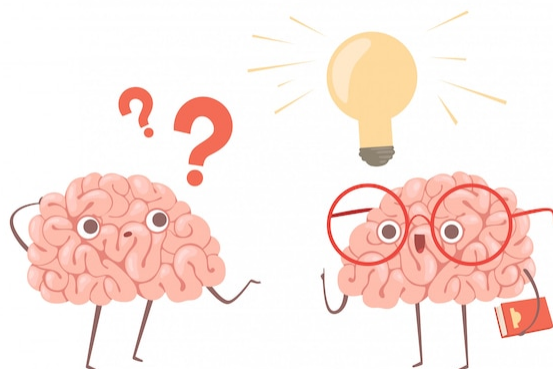
Transformers



Problemas de las RNN

Problema de eficiencia y escalabilidad:

- Son computacionalmente muy costosas de entrenar para secuencias muy largas
- Entrenamiento lento, no permite la paralelización del aprendizaje



Transformers

Idea de transformers

La atención es un mecanismo clave que permite al modelo centrarse en partes relevantes de la entrada y producir una salida contextualizada

Simulando la concentración humana

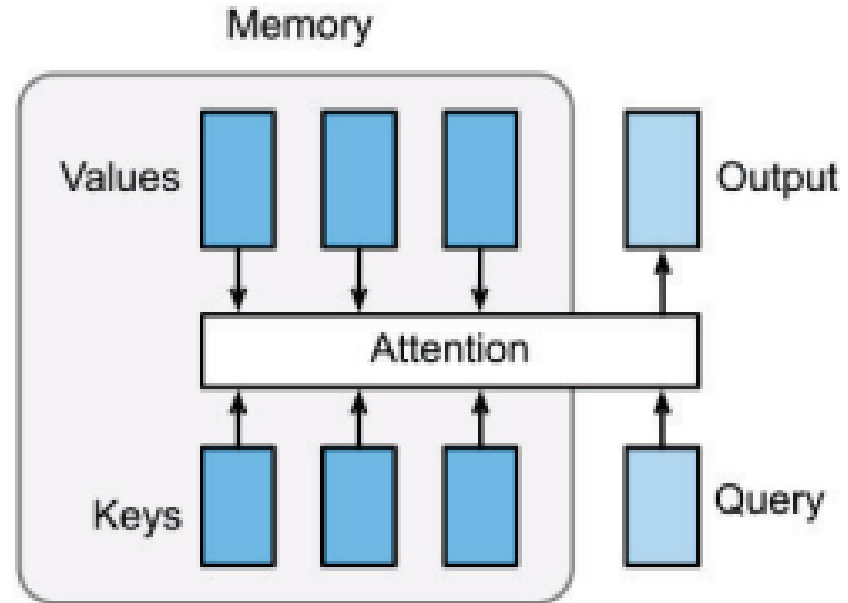
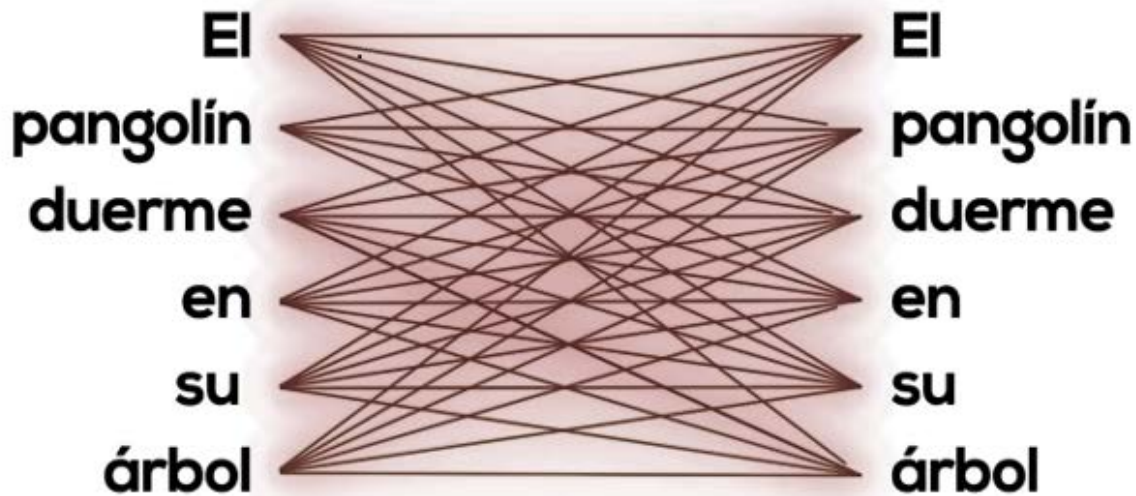


Figura 2.10: (Zhang et al., 2020a)

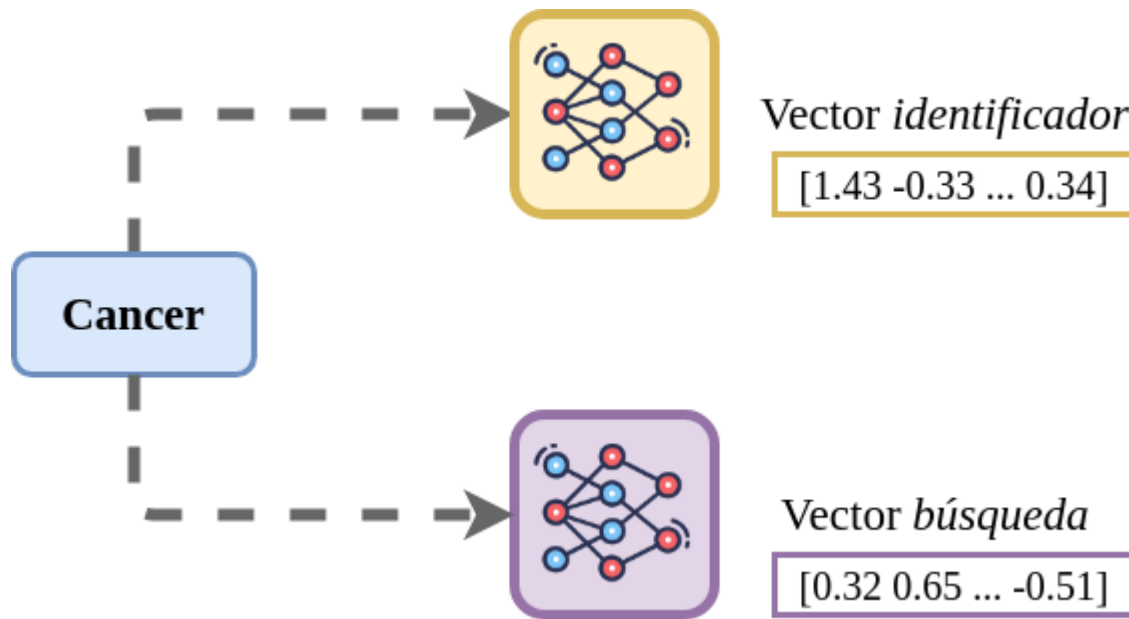
Mecanismo de self-attention



La red aprende la relación de todas las palabras con todas las palabras

Existen palabras con mayor relación contextual como pangolín y árbol (naturaleza)

Mecanismo de self-attention



Vector identificador (key): se utiliza para determinar qué posiciones de la entrada son importantes para una posición dada

Vector búsqueda (query): se utiliza para determinar qué información es relevante para la posición actual

Mecanismo de self-attention

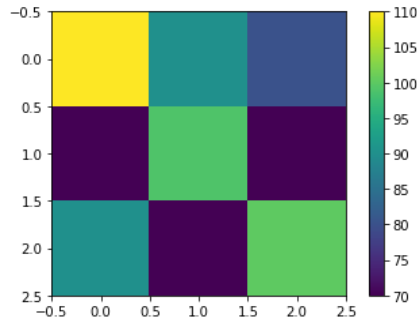
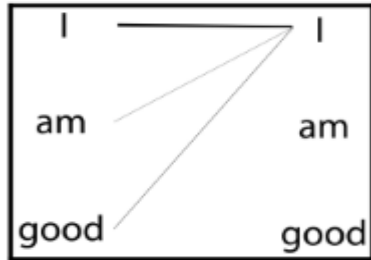
Redes neuronales se encargan de por cada token generar una llave y una cerradura, cada llave puede funcionar en distintas cerraduras con mejor o peor resultado.



Vector Query (Cerradura)
Vector Key (Llave)

Atención = $V_Query \times V_Key$ (Producto escalar)

Mecanismo de self-attention



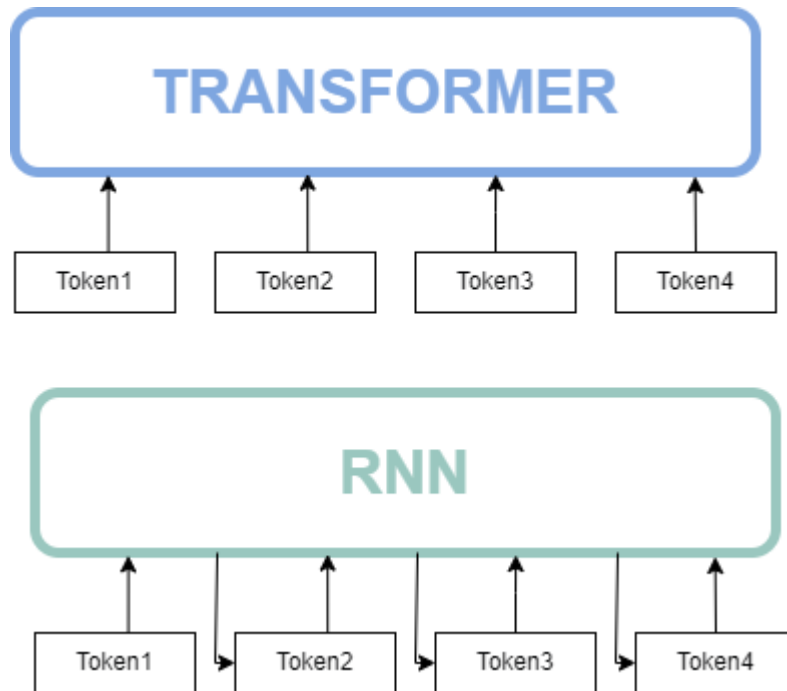
$$\begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} 3.69 & 7.42 & \dots & 4.44 \\ 11.11 & 7.07 & \dots & 76.7 \\ 99.3 & 3.69 & \dots & 0.85 \end{bmatrix} \begin{matrix} q_1 \\ q_2 \\ q_3 \end{matrix} \cdot \begin{array}{c} \text{I} \quad \text{am} \quad \text{good} \\ \begin{bmatrix} 5.31 & 11.71 & 10.10 \\ 6.78 & 0.86 & 11.44 \\ \vdots & \vdots & \vdots \\ 0.96 & 11.31 & 5.11 \end{bmatrix} \\ k_1 \quad k_2 \quad k_3 \end{array} \begin{matrix} k_1 \\ k_2 \\ k_3 \end{matrix}^T = \begin{array}{c} \text{I} \quad \text{am} \quad \text{good} \\ \begin{bmatrix} q_1 \cdot k_1 & q_1 \cdot k_2 & q_1 \cdot k_3 \\ q_2 \cdot k_1 & q_2 \cdot k_2 & q_2 \cdot k_3 \\ q_3 \cdot k_1 & q_3 \cdot k_2 & q_3 \cdot k_3 \end{bmatrix} \end{array}$$

$$QK^T = \begin{array}{c} \text{I} \quad \text{am} \quad \text{good} \\ \begin{bmatrix} 110 & 90 & 80 \\ 70 & 99 & 70 \\ 90 & 70 & 100 \end{bmatrix} \end{array}$$

Matriz de atención = $Q \times K$

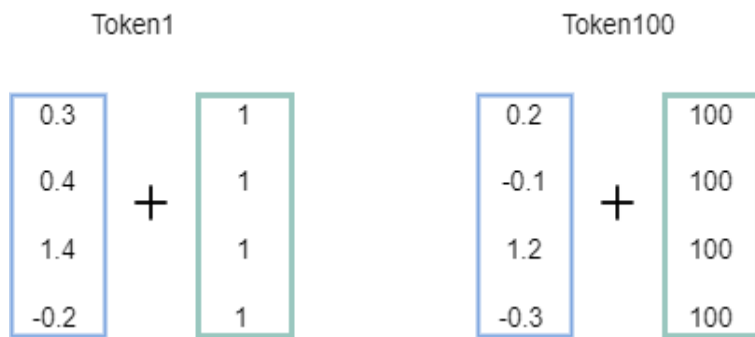
Transformers vs RNN

- En RNN cada palabra tiene que esperar a que se hayan procesado las palabras anteriores
- En cambio los transformers la sentencia se procesa de golpe y por tanto se paraleliza el proceso, pudiendo procesar una mayor cantidad de datos y siendo más eficientes en el entrenamiento
- ¿Cómo se consigue?

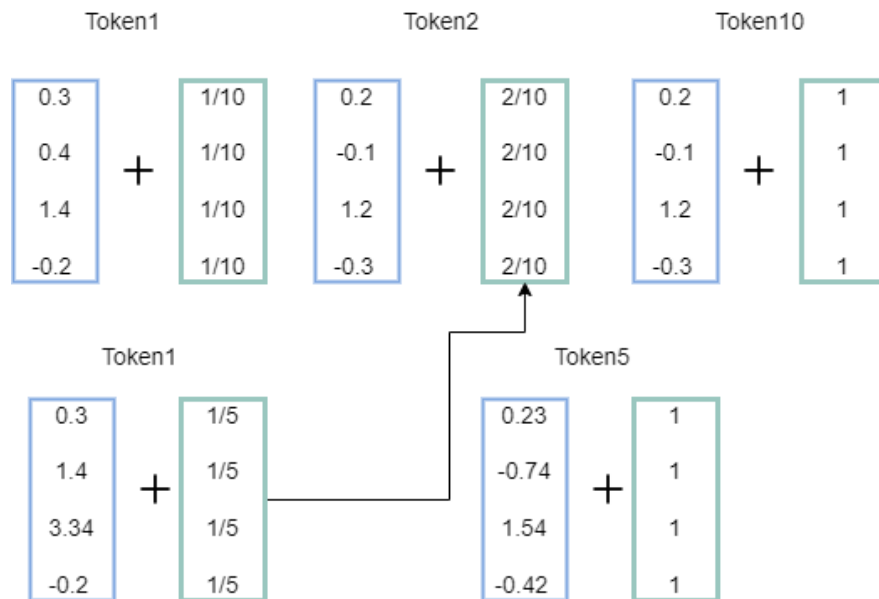


Codificaciones posicionales

Posicionamiento absoluto

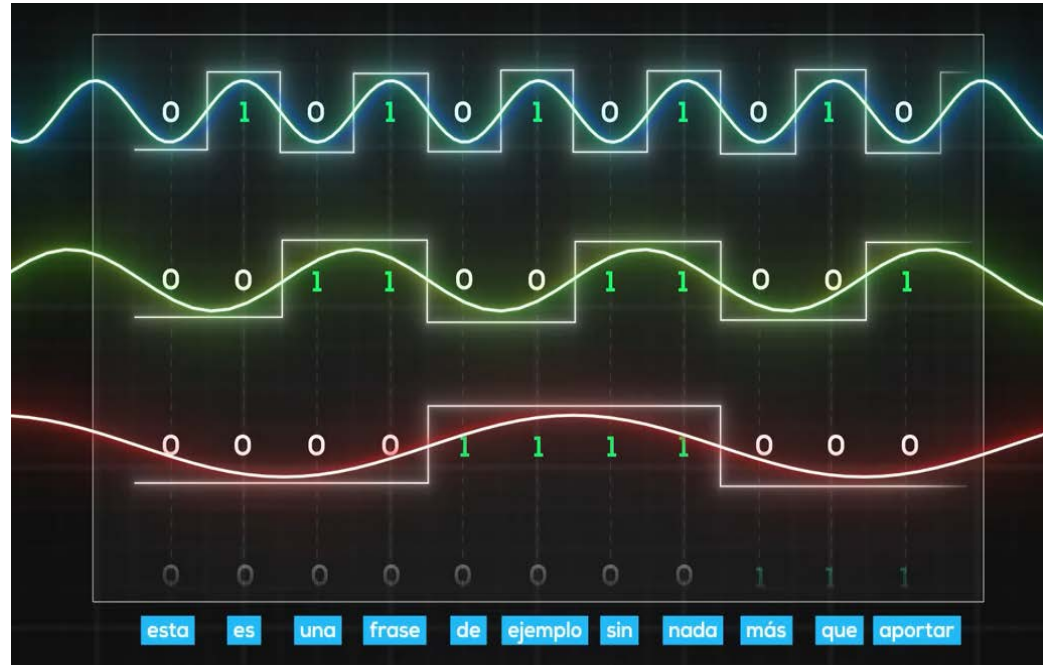


Posicionamiento absoluto normalizado



Codificación binaria

0:	0 0 0 0	8:	1 0 0 0
1:	0 0 0 1	9:	1 0 0 1
2:	0 0 1 0	10:	1 0 1 0
3:	0 0 1 1	11:	1 0 1 1
4:	0 1 0 0	12:	1 1 0 0
5:	0 1 0 1	13:	1 1 0 1
6:	0 1 1 0	14:	1 1 1 0
7:	0 1 1 1	15:	1 1 1 1



Codificación ondulatoria

$$P(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{1000^{2i/d_{\text{model}}}}\right)$$

$$P(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{1000^{2i/d_{\text{model}}}}\right)$$

i = posición en el vector de embedding
 pos = posición en la sentencia
 d_{model} = tamaño vector embedding

$$P = \begin{matrix} & | & \sin\left(\frac{\text{pos}}{10000^0}\right) & \cos\left(\frac{\text{pos}}{10000^0}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \\ \text{am} & & \sin\left(\frac{\text{pos}}{10000^0}\right) & \cos\left(\frac{\text{pos}}{10000^0}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \\ \text{good} & & \sin\left(\frac{\text{pos}}{10000^0}\right) & \cos\left(\frac{\text{pos}}{10000^0}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \end{matrix}$$

$$P = \begin{matrix} & | & \sin(0) & \cos(0) & \sin(0/100) & \cos(0/100) \\ \text{am} & & \sin(1) & \cos(1) & \sin(1/100) & \cos(1/100) \\ \text{good} & & \sin(2) & \cos(2) & \sin(2/100) & \cos(2/100) \end{matrix}$$

Codificación Posicional

	I	1.769	2.22	3.4	5.8	x_1
X =	am	7.3	9.9	8.5	7.1	x_2
	good	9.1	7.1	0.85	10.1	x_3

Input matrix
(embedding matrix)

Figure 1.25 – Input matrix

$$\begin{array}{c}
 \mathbf{X} = \begin{bmatrix} 1.769 & 2.22 & 3.4 & 5.8 \\ 7.3 & 9.9 & 8.5 & 7.1 \\ 9.1 & 7.1 & 0.85 & 10.1 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0.841 & 0.54 & 0.01 & 0.99 \\ 0.909 & -0.416 & 0.02 & 0.99 \end{bmatrix} \\
 \mathbf{X} \qquad \qquad \qquad \mathbf{P} \\
 = \begin{bmatrix} 1.769 & 3.22 & 3.4 & 6.8 \\ 8.14 & 10.44 & 8.51 & 8.09 \\ 10.0 & 6.68 & 0.87 & 11.09 \end{bmatrix}
 \end{array}$$

Figure 1.26 – Adding an input matrix and positional encoding matrix

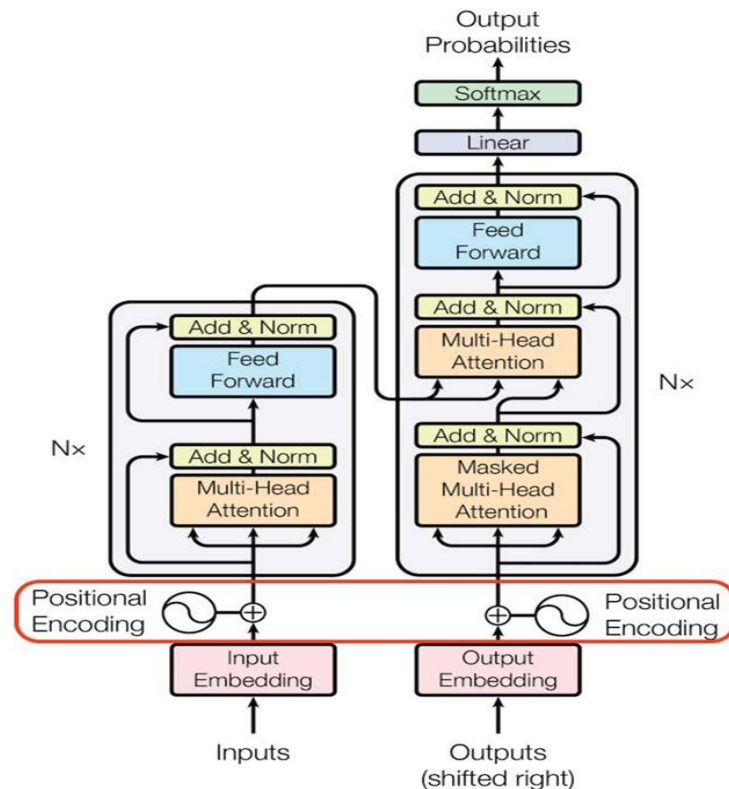
Arquitectura de Transformers

Nueva arquitectura que permite analizar secuencias de datos (píxeles de imágenes, palabras de sentencias o secuencias de aminoácidos) de forma paralela

Input Embedding sigue teniendo la misma funcionalidad convertir las palabras a vectores

Conexiones residuales que evitan el problema del descenso del gradiente

Primera propuesta para traducción $x = 6$, es decir 6 codificadores y 6 decodificadores



“Attention is all you need”

El mecanismo de atención hace la función de memoria

$R = X$ = sentencia codificada como matriz y su posición

$$X \times W^Q = Q$$

$$X \times W^K = K \quad \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V = Z$$

$$X \times W^V = V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

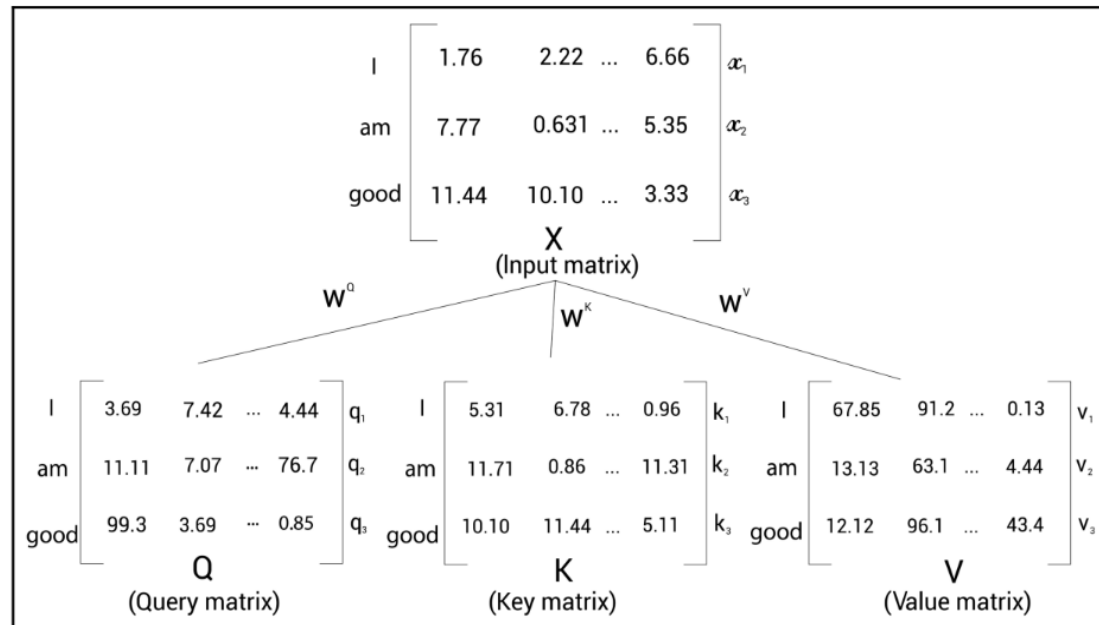
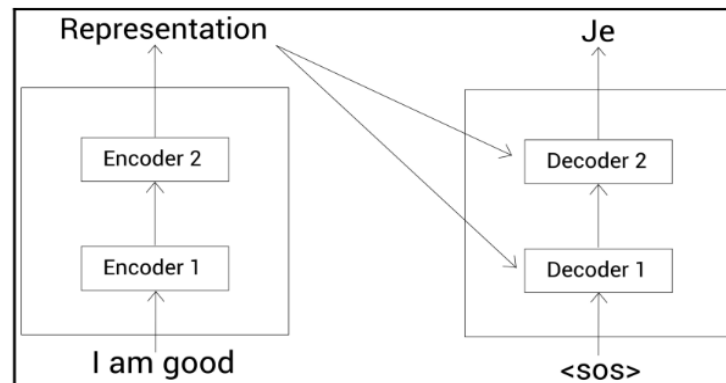
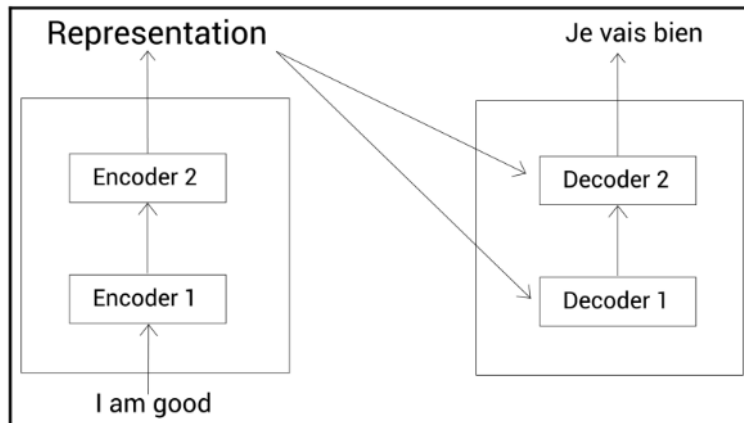
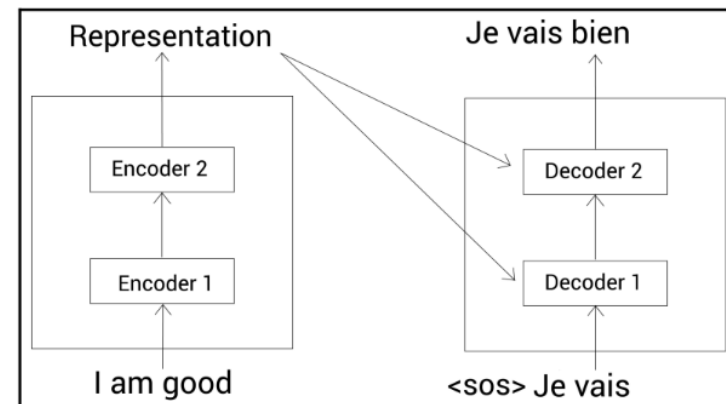


Figure 1.7 – Creating query, key, and value matrices

Traducción con Transformers



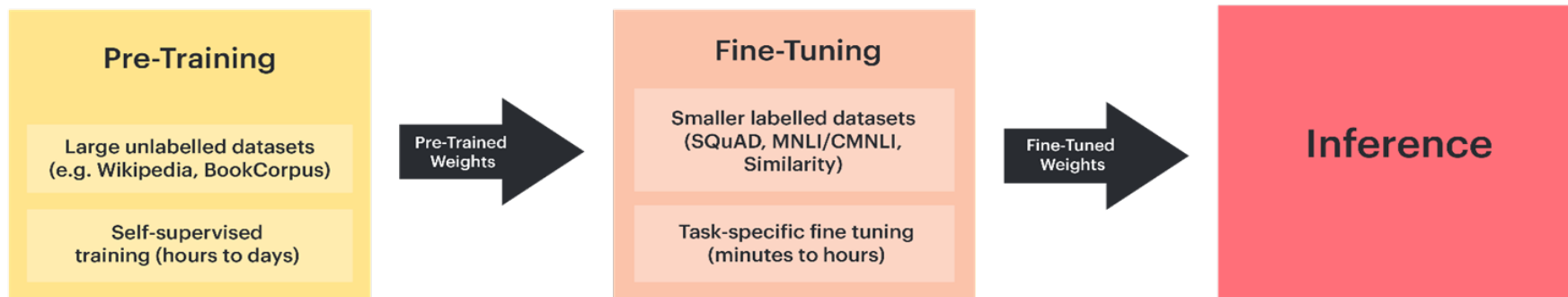
t=0



t=2

Pre-Training & Fine-tuning

El preentrenamiento se enfoca en aprender una representación general del lenguaje.
Fine-Tuning se enfoca en adaptar esa representación a una tarea específica.



El pretraining genera redes como BERT, BETO, o GPT. Existen en la actualidad 174 redes.
(<https://huggingface.co/docs/transformers/index>)

Masked language modeling

tokens = [Paris, is, a beautiful, city, I, love, Paris]

tokens = [[CLS], Paris, is, a beautiful, [MASK], [SEP], I, love, Paris, [SEP]]

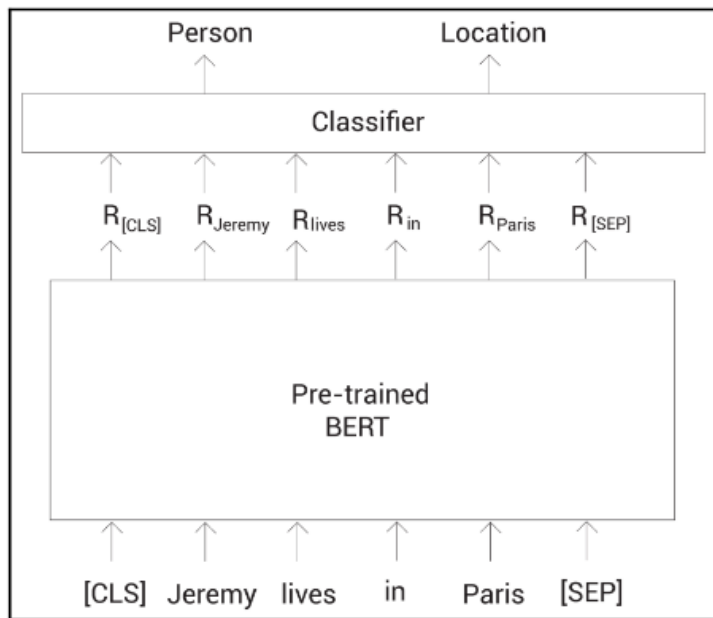
Auto-encoding language modeling

Regla 80-10-10

- 80% se enmascara una palabra (token)
- 10% se reemplaza por un token aleatorio (evitar overfitting)
- 10% se deja el token actual

Existen otras estrategias, como next-sentence-prediction.

Fine-tuning

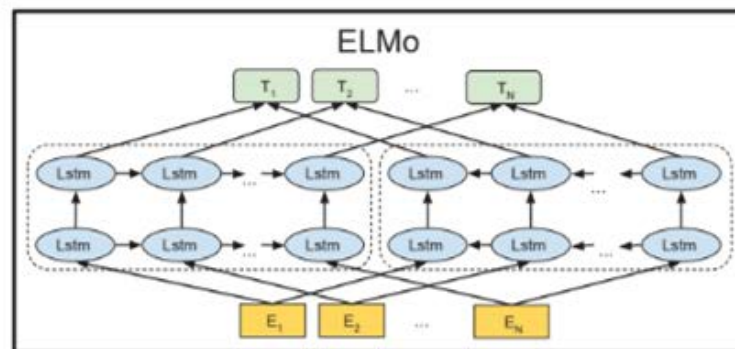
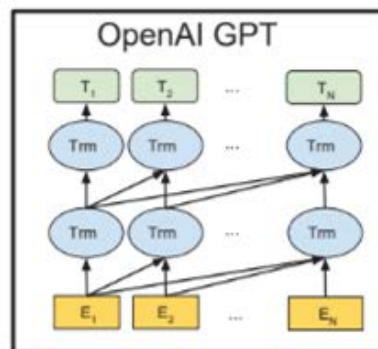
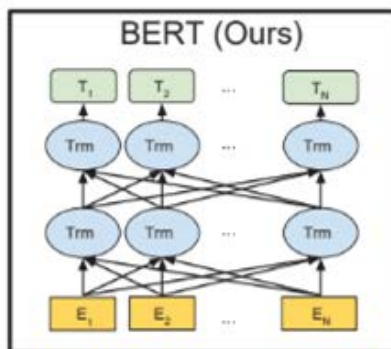


- Detección y traducción de idioma
- Realizar resúmenes
- Análisis de sentimientos
- Reconocimiento de entidades con nombre (NER)
- Extracción de relaciones entre palabras
- Vinculación de entidades (desambiguar)
- Sistemas de Q&A
- Predicción de la siguiente frase (NSP)

BERT

Bidirectional Encoder Representations from Transformers

Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	##ing	[SEP]
Token Embeddings	$E_{[CLS]}$	E_{my}	E_{dog}	E_{is}	E_{cute}	$E_{[SEP]}$	E_{he}	E_{likes}	E_{play}	$E_{\#ing}$	$E_{[SEP]}$
Segment Embeddings	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B
Position Embeddings	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}



NER - Named Entity Recognition



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Corpus conjunto de textos suficientemente grande de muestras reales de una lengua determinada

Corpus anotado Corpus etiquetado (entidades)

evaluacion de la respuesta patologica (miller y payne):

g5 **CANCER_GRADE** ausencia de celularidad tumoral **infiltrante CANCER_EXP** en la **mama CANCER_LOC** y
tipo b **ganglios linfaticos CANCER_LOC** con **metastasis CANCER_MET** y
sin cambios por quimioterapia en valoracion **ganglionar CANCER_LOC** . no **invasion CANCER_MET** **linfovacular CANCER_LOC**

evaluacion de la respuesta patologica (miller y payne):

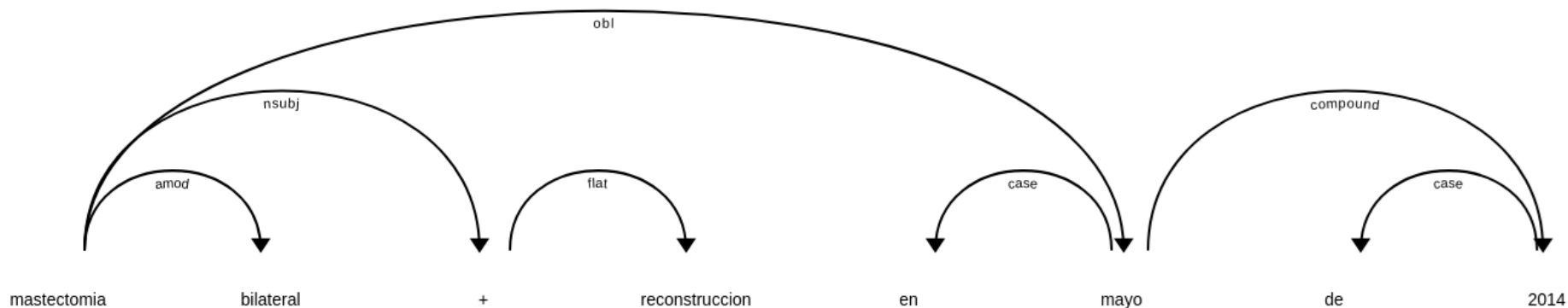
g5 **ausencia de NEG** **celularidad tumoral infiltrante en la mama NSCO** y
tipo b **ganglios linfaticos con metastasis y**

sin NEG **cambios por quimioterapia en valoracion ganglionar NSCO** . **no NEG** **invasion linfovacular NSCO**

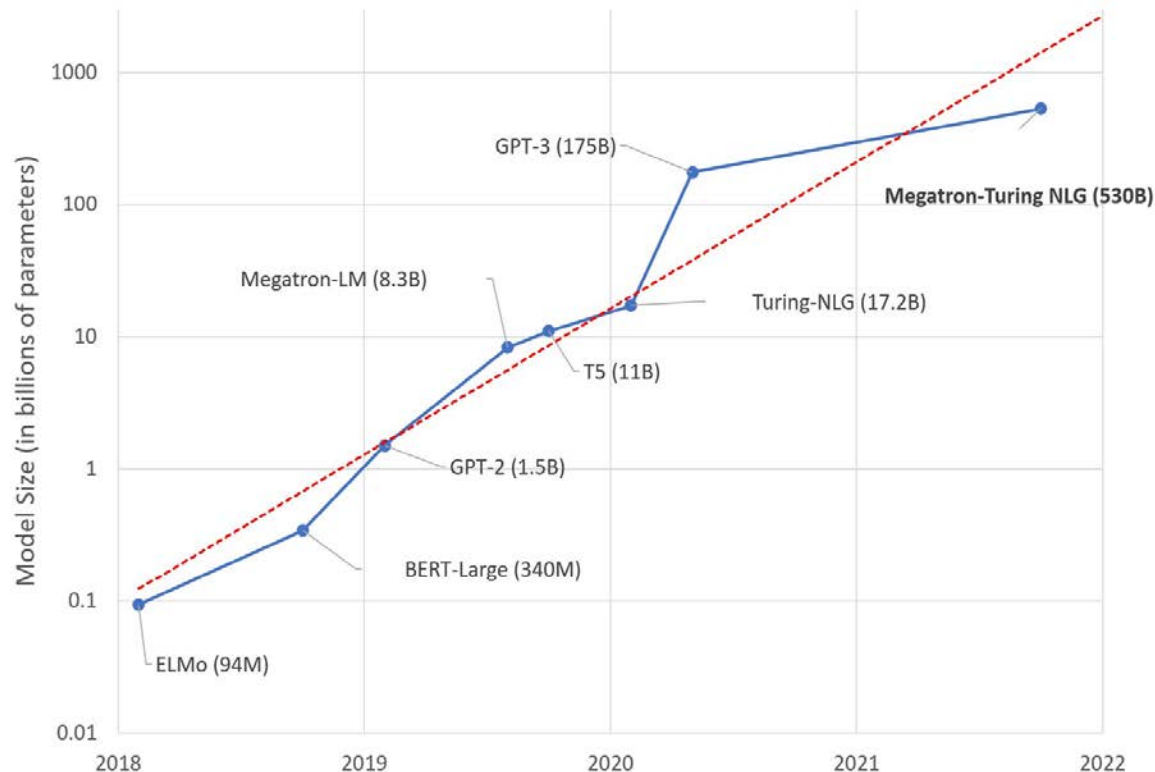
Análisis Sintáctico



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Rendimiento y escalabilidad



Turing-NLG: Microsoft
Megatron-LM: NVIDIA

Megatron-Turing Natural Language Generation model (MT-NLG): el modelo mas potene desarrollado por una colaboracion entre Microsoft y NVIDIA. Entrenado con 530 mil millones de parámetros.

Rendimiento y escalabilidad



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Entrenamiento

GPU

TPU

CPU AVX512

HW especializado

Inferencia

CPU

GPU

TPU

HW especializado

Usos actuales de transformers



UNIVERSIDAD
POLITÉCNICA
DE MADRID

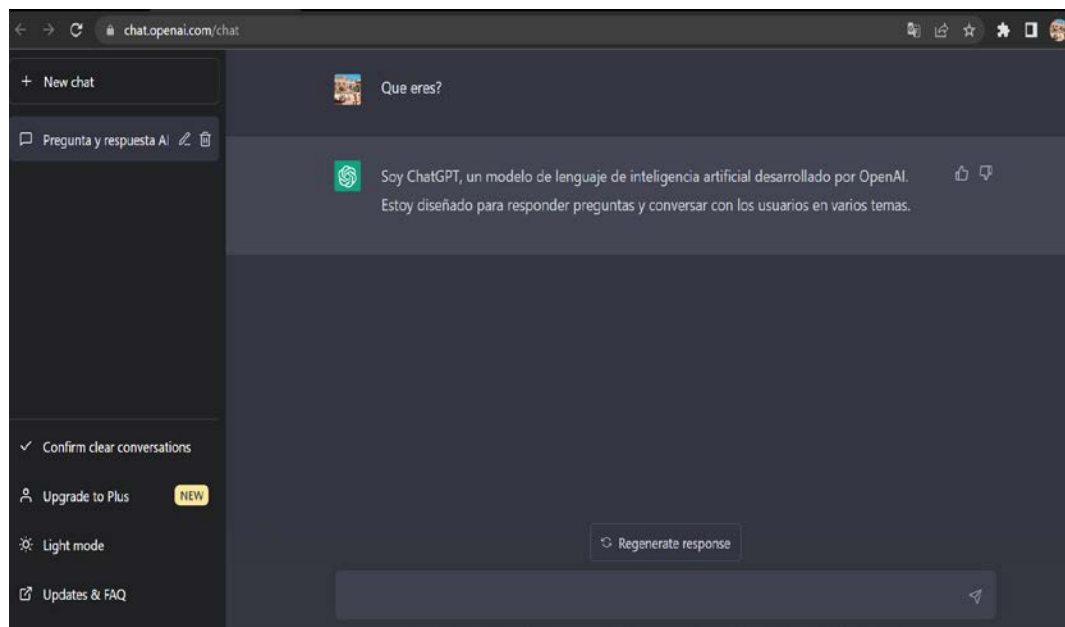


GPT-3.5 (Nov 2022, OpenAI)

Modelo de lenguaje: Generar nuevas secuencias de palabras que tengan una alta probabilidad de ser gramaticalmente correctas y coherentes. (multimodales)

Algunas de las tareas:

- La corrección ortográfica
- La traducción automática
- La generación de texto automático
- La clasificación de texto



Usos actuales de transformers



UNIVERSIDAD
POLITÉCNICA
DE MADRID



DALLE 2 (2022, OpenAI)

- Generar imágenes a partir de descripciones de texto en lenguaje natural.
- Clasificación de imágenes
- Traducción de texto



Usos actuales de transformers



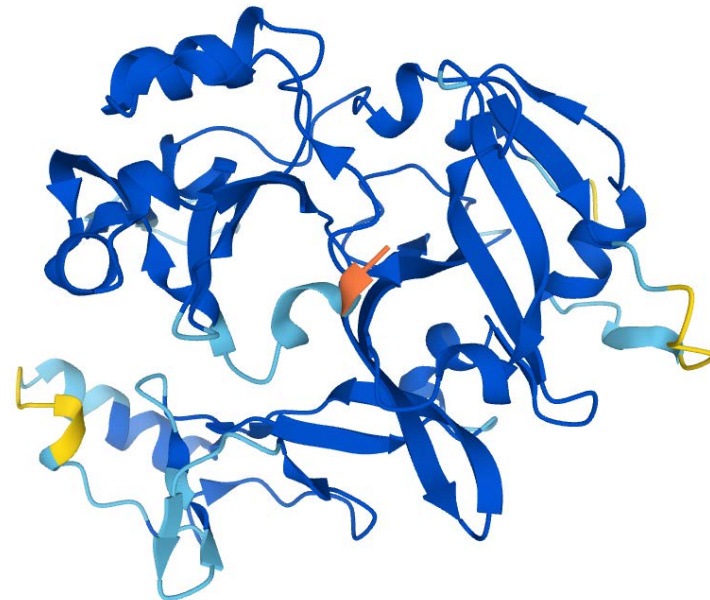
UNIVERSIDAD
POLITÉCNICA
DE MADRID



Alphafold 2 (2020, DeepMind)

Predecir la estructura tridimensional de las proteínas a partir de una secuencia de aminoácidos

Con el objetivo de descubrimiento de nuevos fármacos y tratamientos médicos



Usos actuales de transformers



UNIVERSIDAD
POLITÉCNICA
DE MADRID



MusicLM (2023, Google)

Es un modelo que genera música de alta fidelidad a partir de descripciones de texto



RESEARCH LINES



NATURAL LANGUAGE PROCESSING



IMAGE ANALYSIS

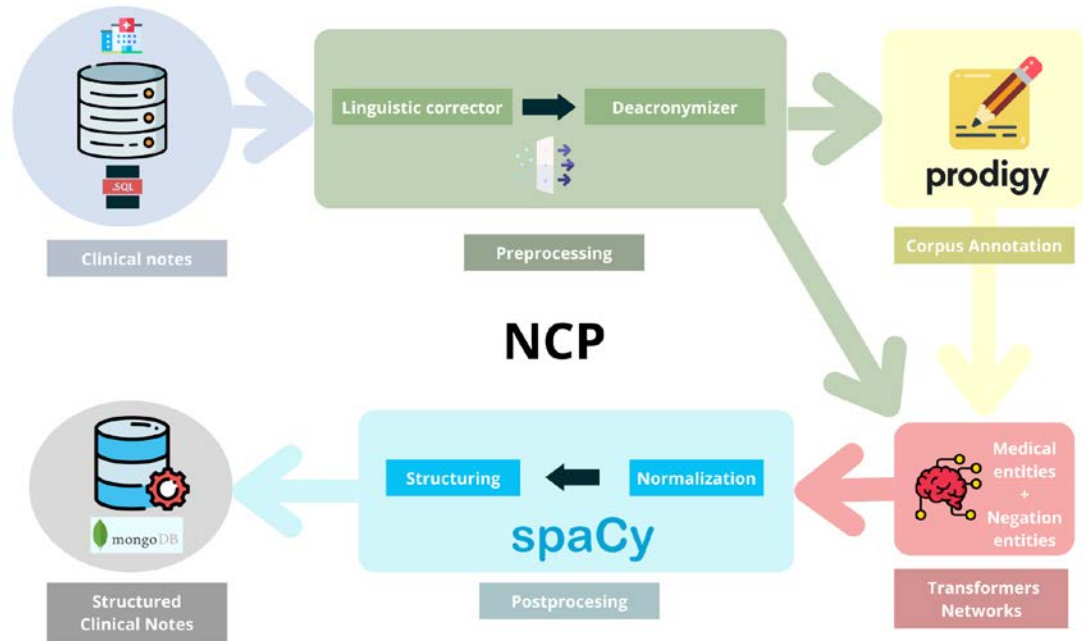


DATA ANALYTICS



DISEASE UNDERSTANDING

TFG NLP Cancer Pipeline - NCP



Publicaciones



UNIVERSIDAD
POLITÉCNICA
DE MADRID



- **Transformers for Extracting Breast Cancer Information from Spanish Clinical Narratives (en revisión)**
Oswaldo Solarte-Pabón, Orlando Montenegro, Alvaro García-Barragán, Maria Torrente, Mariano Provencio, Ernestina Menasalvas, Víctor Robles
Journal: Artificial Intelligence In Medicine, 2022
- **A Deep Learning Approach to Extract Lung Cancer Information from Spanish clinical texts (en revisión)**
Oswaldo Solarte-Pabón, Orlando Montenegro, Álvaro García, Alejandro Rodriguez-González, Juan Cristobal Sanchez, Víctor Robles, Mariano Provencio, and Ernestina Menasalvas
2022
- **Deep learning to extract Breast Cancer diagnosis concepts (publicado)**
Oswaldo Solarte-Pabón, María Torrente, Álvaro García-Barragán, Mariano Provencio, Ernestina Menasalvas, Víctor Robles
2022 IEEE 35th International Symposium on Computer-Based Medical Systems (CBMS)
- **Structuring Breast Cancer Spanish Electronic Health Records Using Deep Learning (en preparación)**
Álvaro García-Barragán, Oswaldo Solarte-Pabón, Mariano Provencio, Ernestina Menasalvas, Víctor Robles
2023 IEEE 36th International Symposium on Computer-Based Medical Systems (CBMS)
- **Dropout Prediction through Machine Learning in Higher Education (en revisión)**
Genaro Zavala, Claudia Lizette Garay-Rondero, Alvaro Garcia-Barragan, Andres E. Gutierrez-Rodriguez, Ernestina Menasalvas, Victor Robles
Journal: Computers & Education, 2023

Bibliografía



UNIVERSIDAD
POLITÉCNICA
DE MADRID



- Resumen de textos literarios en español por medio de modelos lingüísticos Transformers, Guillermo Marco Remón, TFM, UPM.
- Attention is all you need, A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, { . Kaiser, and I. Polosukhin. Advances in Neural Information Processing Systems , page 5998--6008. (2017)
- Getting Started with Google BERT: Build and train state-of-the-art natural language processing models using BERT.
- Transformers for Natural Language Processing: Build, train, and fine-tune deep neural network architectures for NLP with Python, PyTorch, TensorFlow, BERT, and GPT-3.
- Canal de youtube de Carlos Santana Vega, divulgador científico de Inteligencia Artificial

¡Muchas gracias!

Álvaro García Barragán
Facultad de Informática, Universidad Complutense de Madrid
alvaga28@ucm.es



UNIVERSIDAD
POLITÉCNICA
DE MADRID



UNIVERSIDAD
COMPLUTENSE
MADRID